

Labs UD

Guía técnica de desarrollo, despliegue y operación

Documento interno (para administración del proyecto)

21 de enero de 2026

Propósito: este documento explica (de forma práctica) cómo está construido el sistema **Labs UD**, cómo se conecta a la base de datos, cómo se administra desde el panel de administrador y cómo se despliega en producción (Render) desde GitHub.

Índice

1. Descripción general del sistema	3
2. Tecnologías y componentes usados	3
2.1. Backend	3
2.2. Base de datos	3
2.3. Despliegue	3
3. Repositorio y flujo de Git (pull / commit / push)	3
3.1. Estructura general	3
4. Base de datos en Neon y acceso con pgAdmin 4	4
4.1. Neon (PostgreSQL en la nube)	4
4.2. ¿Dónde está la cadena de conexión?	4
4.3. Acceso por pgAdmin 4	4
5. Arquitectura MVC y piezas del código	4
5.1. Program.cs (arranque y middlewares)	4
5.2. DbContext (Data/AppDbContext.cs)	5
5.3. Modelos principales (Models/)	5
5.4. Controladores (Controllers/)	5
6. Roles y metodología de ingreso	6
6.1. Roles	6
6.2. Pantalla de ingreso (captura de datos)	6
7. Panel de administración (módulos)	7
7.1. Laboratorios y bloqueo de franjas (clases)	7

7.2. Bancos	8
7.3. Reservas (vista administrativa)	9
7.4. Estudiantes registrados	9
7.5. Usuarios / Administradores	9
8. Despliegue en Render con Docker	10
8.1. Dockerfile (build y runtime)	10
8.2. Errores comunes en Render (lo que se evidenció en el despliegue)	10
9. Mapa rápido: pantallas ↔ controlador/acción	10
10. Estructura de la base de datos	11
10.1. Tablas principales	11
10.1.1. Tabla laboratorio	11
10.1.2. Tabla banco	12
10.1.3. Tabla estudiante	12
10.1.4. Tabla usuarios	12
10.1.5. Tabla franja_horaria	13
10.1.6. Tabla reserva_banco	13
10.2. Relaciones entre tablas	13

1. Descripción general del sistema

Labs UD es una aplicación web en **ASP.NET Core 8 (MVC)** para gestionar:

- **Laboratorios:** registro y mantenimiento de laboratorios.
- **Bancos:** registro de bancos asociados a un laboratorio (capacidad, estado, etc.).
- **Reservas:** reservas de bancos por parte de estudiantes y consulta/seguimiento por administradores.
- **Usuarios y estudiantes:** gestión de perfiles (incluyendo asignación de rol administrador).
- **Franjas horarias:** definición de franjas y bloqueo de franjas recurrentes (por clases) en un laboratorio.

El proyecto se desarrolló localmente (Visual Studio), se versionó en GitHub y se desplegó en Render usando un **Dockerfile**.

2. Tecnologías y componentes usados

2.1. Backend

- **.NET 8 + ASP.NET Core MVC** (controladores, vistas Razor, routing).
- **Entity Framework Core** como ORM.
- **Npgsql** como proveedor de PostgreSQL.
- **Sesiones** para mantener el usuario autenticado y su rol (admin/no admin).

2.2. Base de datos

- **PostgreSQL en Neon** (DB en la nube).
- **pgAdmin 4** para administración/inspección de tablas, consultas y datos.

2.3. Despliegue

- **GitHub:** repositorio remoto del código.
- **Render:** despliegue automático desde GitHub mediante Docker.
- **Docker:** imagen basada en runtime ASP.NET 8.

3. Repositorio y flujo de Git (pull / commit / push)

3.1. Estructura general

El repositorio contiene un proyecto MVC típico:

- **Controllers/**: lógica HTTP (acciones GET/POST).
- **Models/**: entidades (tablas) y modelos auxiliares.
- **Views/**: pantallas (Razor) organizadas por controlador.
- **Data/**: contexto EF Core (DbContext) y configuración de relaciones.
- **wwwroot/**: recursos estáticos (CSS, JS, imágenes).
- **Program.cs**: configuración del pipeline (MVC, sesiones, rutas).
- **Dockerfile**: empaquetado para producción.

4. Base de datos en Neon y acceso con pgAdmin 4

4.1. Neon (PostgreSQL en la nube)

Neon ofrece una instancia PostgreSQL accesible por red (host/puerto). La aplicación se conecta usando una **cadena de conexión**.

4.2. ¿Dónde está la cadena de conexión?

La referencia principal está en:

- `appsettings.json` → `ConnectionStrings:DefaultConnection`

En producción, Render también permite definir la cadena como **variable de entorno**. Si se usa variable de entorno, su ubicación es:

- Panel del servicio en Render → `Environment(Variables)`

4.3. Acceso por pgAdmin 4

Para validar tablas/datos:

1. Crear un **Server** en pgAdmin.
2. Configurar *Host*, *Port*, *Database*, *Username* y SSL según Neon.
3. Probar conexión y ejecutar consultas como `SELECT * FROM public.laboratorio;` o `SELECT * FROM public.banco;`

5. Arquitectura MVC y piezas del código

5.1. Program.cs (arranque y middlewares)

El archivo `Program.cs` configura:

- Registro de MVC: `AddControllersWithViews()`.

- Registro de **DbContext** (EF Core + PostgreSQL).
- Habilitación de **sesiones** (para mantener login/rol).
- Pipeline: `UseStaticFiles`, `UseRouting`, `UseSession`, `UseAuthorization`.
- Ruta por defecto: `Account/Login`.

5.2. DbContext (Data/AppDbContext.cs)

`AppDbContext` es el punto central que conecta EF Core con la BD. Define `DbSet<>` para:

- `Estudiantes`, `Laboratorios`, `Bancos`, `FranjasHorarias`, `ReservasBanco`.

También define relaciones (por ejemplo, un *Banco* pertenece a un *Laboratorio*, y una *Reserva* se asocia a *Banco*, *Estudiante* y *FranjaHoraria*).

5.3. Modelos principales (Models/)

- **Estudiante**: datos base del perfil (código, nombre, email, programa) y bandera de rol admin.
- **Laboratorio**: datos descriptivos (nombre, sede, piso, observaciones).
- **Banco**: código, capacidad, estado, laboratorio asociado.
- **FranjaHoraria**: nombre y rango horario (inicio/fin).
- **ReservaBanco**: reserva (fecha, estado) con llaves foráneas a estudiante/banco/franja.

5.4. Controladores (Controllers/)

A nivel funcional, cada controlador se encarga de un módulo:

Controlador	Responsabilidad principal
<code>AccountController</code>	Gestión del flujo de ingreso al sistema: validación de datos del usuario, creación de sesión, identificación del rol (estudiante o administrador) y redirección a la vista correspondiente.
<code>AdminController</code>	Acceso al panel principal de administración, desde donde se centraliza la navegación hacia los distintos módulos administrativos.
<code>LaboratoriosController</code>	Gestión completa de laboratorios (crear, editar y eliminar), así como la administración de bloqueos de franjas horarias recurrentes para clases.
<code>BancosController</code>	Administración de los bancos asociados a cada laboratorio, incluyendo creación, edición, eliminación y control de estado.

Controlador	Responsabilidad principal
ReservasController	Gestión de reservas de bancos por parte de los estudiantes y visualización administrativa del historial y estado de las reservas.
AdminEstudiantesController	Gestión de estudiantes registrados: listado, creación manual, edición de datos personales y asignación o retiro de permisos administrativos.
AdminUsuariosController	Administración de usuarios con rol administrador, permitiendo el control de accesos y permisos del sistema.

6. Roles y metodología de ingreso

6.1. Roles

Estudiante (no admin):

- Puede iniciar sesión.
- Puede ver su módulo de reservas (mis reservas) y reservar bancos.

Administrador:

- Accede al panel de administración.
- Gestiona laboratorios, bancos, reservas, estudiantes y usuarios/administradores.

6.2. Pantalla de ingreso (captura de datos)

The screenshot shows a web application interface for 'Labs UD'. At the top, there's a header with 'Labs UD' and 'Mis Reservas' on the left, and 'Ingresar' on the right. The main content area is titled 'Ingreso a laboratorios'. Below this title, there's a light blue box with the heading 'Antes de continuar' and a message: 'Si es la primera vez que ingresas a este sistema, se creará tu perfil con los datos que escribas a continuación. Por favor:'. It lists three bullet points: 'Escribe tu nombre completo en MAYÚSCULAS', 'Verifica muy bien tu código estudiantil (11 dígitos)', and 'Usa tu correo institucional (@udistrital.edu.co)'. Below this box, it says 'Estos datos quedarán guardados y se usarán para tus futuras reservas de laboratorio.' and there's a red button that says 'He leído y deseo continuar'. Below the instructions, there are three input fields: 'Código de estudiante' (with the value '20151005121'), 'Nombre completo' (with the value 'Yeison'), and 'Correo institucional (@udistrital.edu.co)' (with the value 'ysgutierrezm@udistrital.edu.co'). At the bottom of the form is a red button labeled 'Ingresar'. The browser's address bar at the bottom shows 'https://localhost:7116/Reservas/MisReservas'.

Figura 1: Pantalla de ingreso: captura de código, nombre y correo institucional.

Qué hace esta pantalla:

- Solicita *Código, Nombre y Email*.
- Si el estudiante no existe, se crea su registro.
- Si existe, se usa para iniciar sesión.
- Se guarda la información necesaria en **sesión** (para navegación y permisos).

7. Panel de administración (módulos)

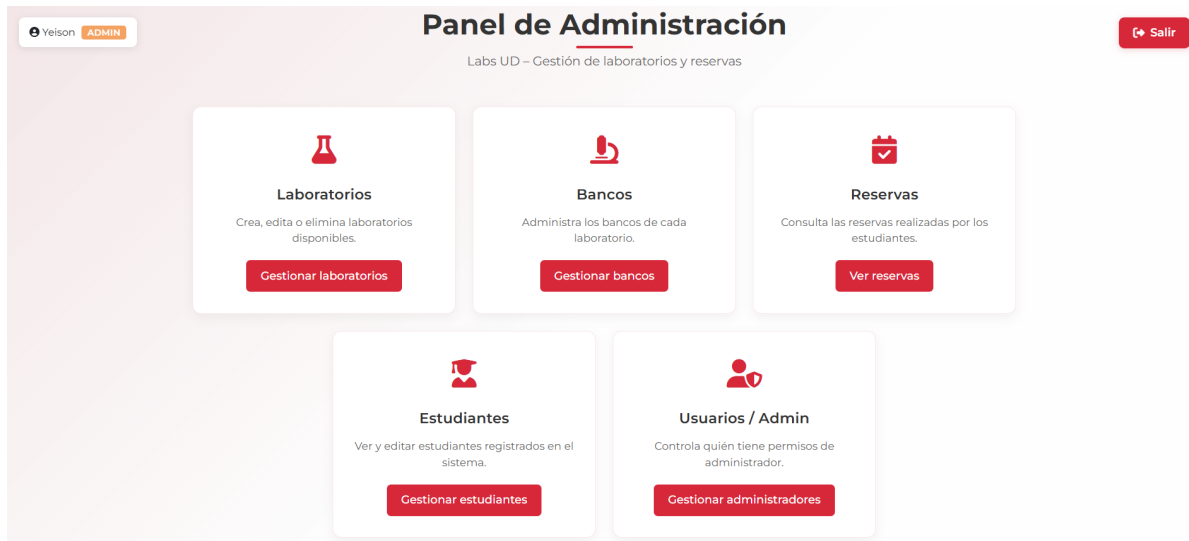


Figura 2: Panel principal de administración (menú).

Desde este panel se navega a:

- **Laboratorios**
- **Bancos**
- **Reservas**
- **Estudiantes**
- **Usuarios / Administradores**

7.1. Laboratorios y bloqueo de franjas (clases)

Qué resuelve este módulo:

- CRUD de laboratorios.
- Definir **ocupaciones semanales** (por ejemplo, clases) para evitar reservas en esas franjas.
- Mostrar ocupaciones registradas y permitir quitarlas.

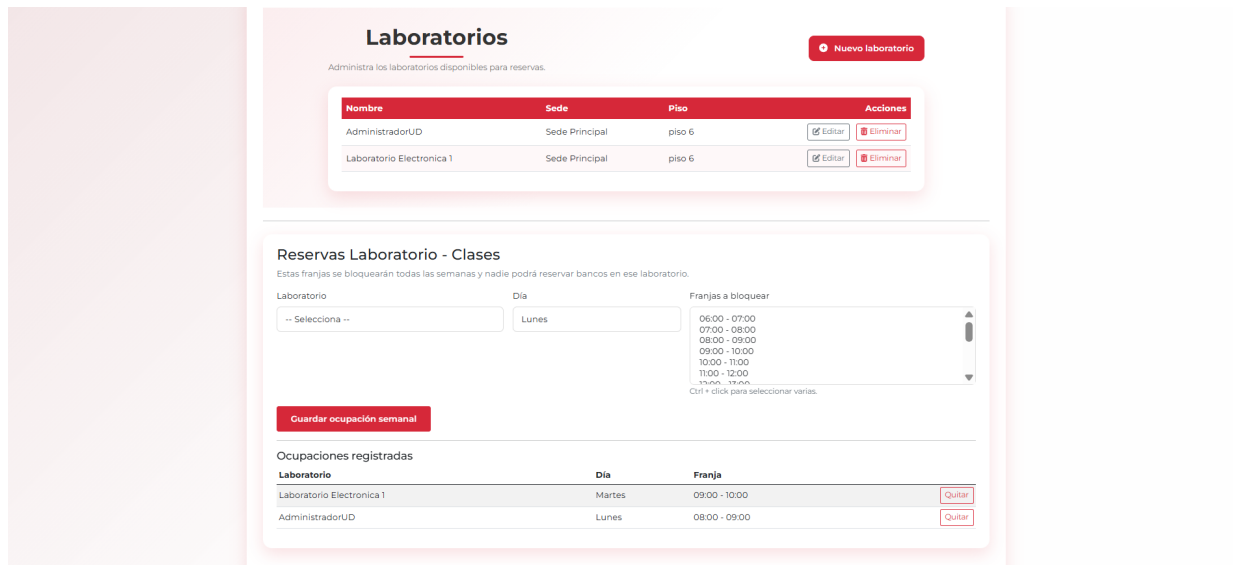


Figura 3: Gestión de laboratorios y sección de bloqueo de franjas recurrentes.

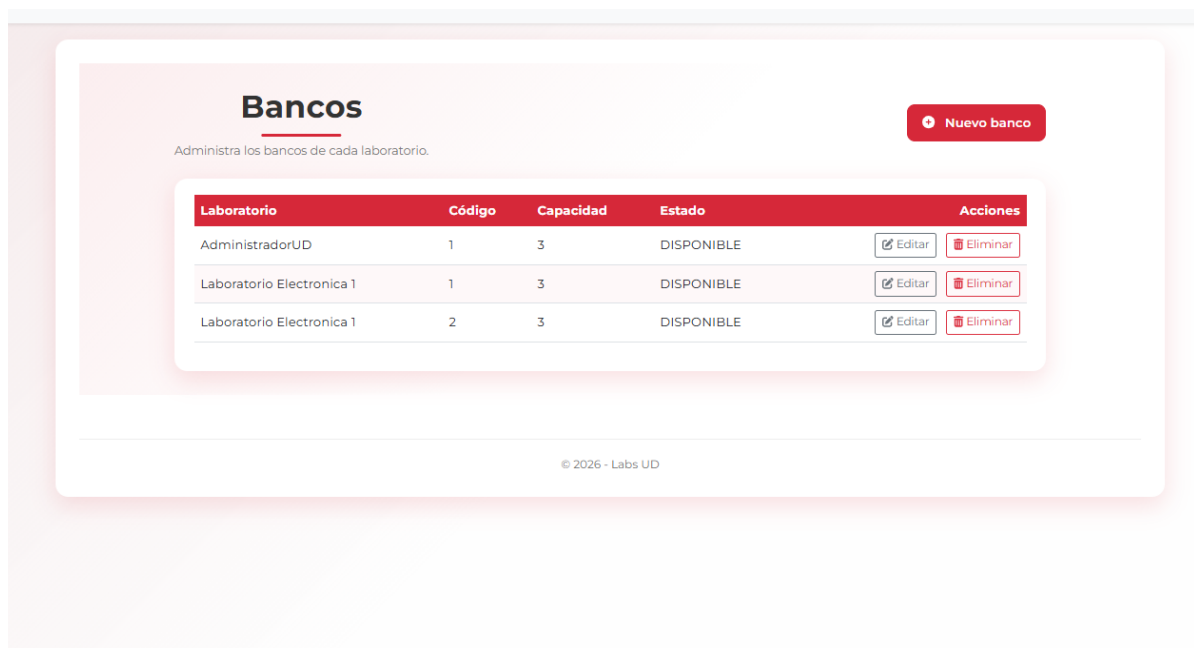


Figura 4: Gestión de bancos por laboratorio.

7.2. Bancos

Qué resuelve este módulo:

- Crear bancos para cada laboratorio (código, capacidad).
- Cambiar estado (por ejemplo *DISPONIBLE*).
- Editar/eliminar bancos.

7.3. Reservas (vista administrativa)

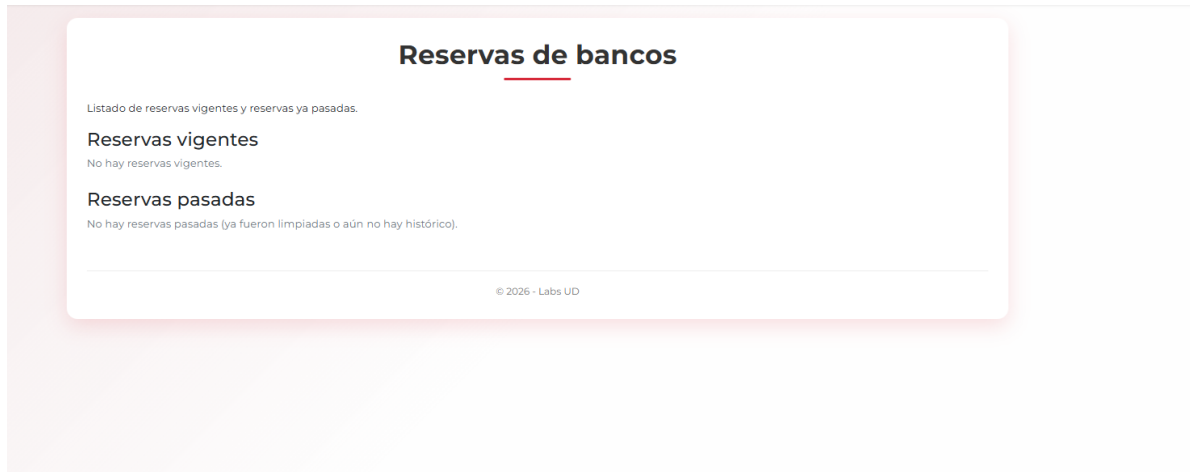


Figura 5: Listado de reservas desde el perfil administrador.

En esta vista se separan *reservas vigentes* y *reservas pasadas*. Sirve para auditoría y control del uso del laboratorio.

7.4. Estudiantes registrados

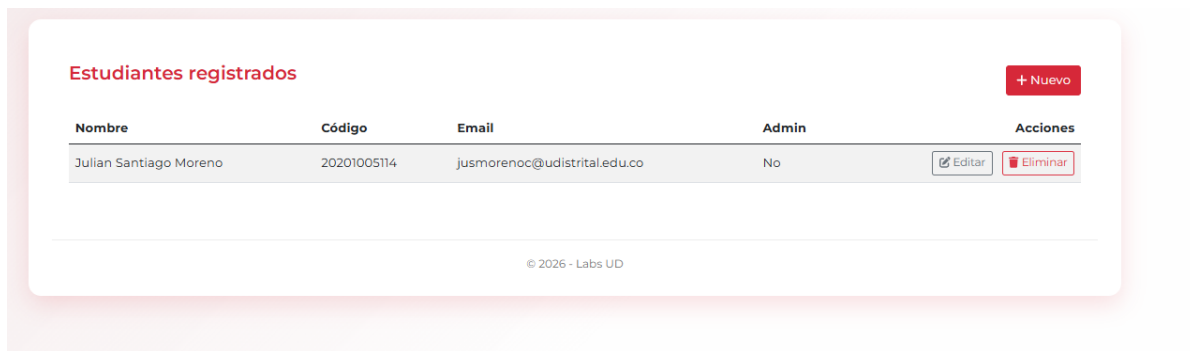


Figura 6: Gestión de estudiantes: listado y acciones.

Operaciones típicas:

- Ver estudiantes registrados.
- Editar datos (nombre, código, correo, programa).
- Convertir/retirar permiso de administrador (según check de rol).

7.5. Usuarios / Administradores

Este módulo centraliza la administración de quién tiene permisos elevados.

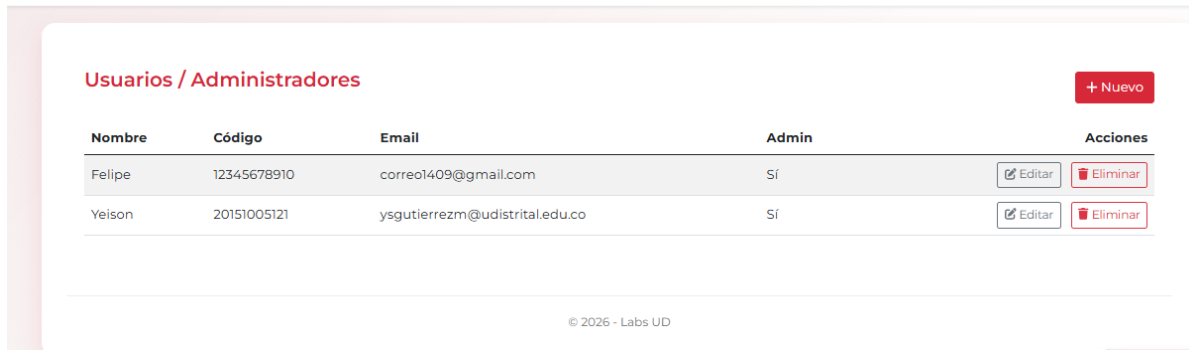


Figura 7: Gestión de usuarios con rol administrador.

8. Despliegue en Render con Docker

8.1. Dockerfile (build y runtime)

El despliegue se hace con un Dockerfile en dos etapas:

- **build:** usa `mcr.microsoft.com/dotnet/sdk:8.0` para restaurar dependencias y publicar.
- **final:** usa `mcr.microsoft.com/dotnet/aspnet:8.0` para ejecutar el binario publicado.

El contenedor expone el puerto **8080** y define `ASPNETCORE_URLS=http://+:8080`.

8.2. Errores comunes en Render (lo que se evidenció en el despliegue)

- **HTTP 500 después de enviar un formulario:** normalmente está relacionado con una excepción en servidor (validación, acceso a DB, sesión, etc.). La revisión se hace en los *logs* del servicio.
- **“Failed to determine the https port for redirect”:** Render termina TLS, por lo que el contenedor escucha HTTP interno. Si el proyecto fuerza redirección HTTPS sin configurar puerto, aparece esa advertencia.
- **Límites de inotify / FileSystemWatcher:** puede ocurrir si el runtime intenta “vigilar” muchos archivos. En contenedores se evita usando configuración de producción (sin `reload-on-change`).

9. Mapa rápido: pantallas ↔ controlador/acción

Este mapa sirve como guía para ubicar la lógica en el código cuando se requiera mantenimiento.

Pantalla / Función	Ubicación típica en el proyecto
Login / ingreso	AccountController + Views/Account/Login.cshtml

Panel admin	AdminController	+
	Views/Admin/Index.cshtml	
CRUD laboratorios	LaboratoriosController	+
	Views/Laboratorios/*	
Bloqueo semanal de franjas	LaboratoriosController (acción dedicada)	+
	vista integrada	
CRUD bancos	BancosController + Views/Bancos/*	
Reservas (usuario)	ReservasController + Views/Reservas/*	
Reservas (admin)	ReservasController (acción admin) + vista admin	
CRUD estudiantes	AdminEstudiantesController	+
	Views/AdminEstudiantes/*	
CRUD usuarios/admin	AdminUsuariosController	+
	Views/AdminUsuarios/*	

10. Estructura de la base de datos

El sistema **Labs UD** utiliza una base de datos relacional **PostgreSQL**, alojada en la plataforma **Neon**, la cual permite una conexión remota segura y persistente desde el aplicativo desplegado en Render.

La base de datos fue diseñada siguiendo un modelo relacional normalizado, con el objetivo de:

- Garantizar integridad de la información.
- Evitar duplicidad de datos.
- Facilitar el control de roles (estudiantes y administradores).
- Soportar el flujo de reservas de bancos por franja horaria.

10.1. Tablas principales

A continuación se describen las tablas que conforman el núcleo del sistema y su propósito dentro de la aplicación.

10.1.1. Tabla laboratorio

Esta tabla almacena la información general de los laboratorios disponibles para reservas.

- **id_laboratorio**: Identificador único del laboratorio (clave primaria).
- **nombre**: Nombre del laboratorio (por ejemplo: Laboratorio Electrónica 1).
- **sede**: Ubicación física del laboratorio.
- **piso**: Piso donde se encuentra el laboratorio.

- **observaciones:** Información adicional relevante.

Cada laboratorio puede tener asociados múltiples bancos y múltiples restricciones horarias.

10.1.2. Tabla banco

Representa los puestos físicos (bancos) dentro de cada laboratorio.

- **id_banco:** Identificador único del banco (clave primaria).
- **id_laboratorio:** Relación con la tabla laboratorio (clave foránea).
- **codigo:** Código interno del banco.
- **capacidad:** Número de estudiantes permitidos.
- **estado:** Estado del banco (DISPONIBLE / OCUPADO).
- **observaciones:** Notas adicionales.

Un laboratorio puede contener múltiples bancos, pero cada banco pertenece a un solo laboratorio.

10.1.3. Tabla estudiante

Almacena los datos de los estudiantes que han ingresado al sistema.

- **id_estudiante:** Identificador único del estudiante.
- **codigo:** Código estudiantil institucional (11 dígitos).
- **nombre:** Nombre completo del estudiante.
- **email:** Correo institucional.
- **programa:** Programa académico.
- **es_admin:** Indicador booleano que define si el estudiante tiene rol administrativo.

Esta tabla permite que un estudiante pueda ser promovido a administrador sin duplicar registros.

10.1.4. Tabla usuarios

Contiene a los usuarios con permisos administrativos explícitos.

- **id:** Identificador único.
- **codigo:** Código institucional.
- **nombre:** Nombre del administrador.
- **email:** Correo electrónico.
- **es_admin:** Campo booleano siempre verdadero.

Esta separación permite controlar permisos administrativos de forma clara desde el panel de administración.

10.1.5. Tabla `franja_horaria`

Define los intervalos de tiempo disponibles para reservas.

- **id_franja**: Identificador único.
- **nombre**: Etiqueta de la franja (ej. 08:00 - 09:00).
- **hora_inicio**: Hora de inicio.
- **hora_fin**: Hora de finalización.

Las franjas se generan para:

- Lunes a viernes: 06:00 a 22:00.
- Sábados: 06:00 a 14:00.

10.1.6. Tabla `reserva_banco`

Es la tabla central del sistema, donde se registran todas las reservas.

- **id_reserva**: Identificador único.
- **id_estudiante**: Relación con el estudiante.
- **id_banco**: Banco reservado.
- **id_franja**: Franja horaria asignada.
- **fecha**: Fecha de la reserva.
- **estado**: Estado de la reserva (ACTIVA / FINALIZADA).

Esta tabla permite:

- Mostrar reservas vigentes.
- Mantener historial de reservas pasadas.
- Controlar disponibilidad en tiempo real.

10.2. Relaciones entre tablas

- Un **laboratorio** tiene muchos **bancos**.
- Un **banco** puede tener muchas **reservas**.
- Un **estudiante** puede realizar múltiples **reservas**.
- Cada **reserva** está asociada a una única **franja horaria**.

Esta estructura garantiza consistencia, escalabilidad y facilidad de mantenimiento del sistema.